

Exercice 1 : QCM (0,5 x 10 = 5 pts)

Cochez la bonne réponse (une seule réponse est correcte pour chaque question) :

1. Les applications communiquent avec le noyau en utilisant ?

- ☐ Le shell
- ☒ Des appels système
- ☐ Les processus
- ☐ Un programme C

2. Dans un système d'exploitation, un processus est :

- ☒ L'instance d'un programme en cours d'exécution.
- ☐ Une opération d'Entrée/Sortie.
- ☐ Un utilisateur connecté au système et utilisant des ressources.
- ☐ Un fichier statique stocké sur une mémoire de masse.

3. Dans un SE, l'ordonnanceur (scheduler) est chargé de :

- ☐ Ordonnancer l'utilisation de la mémoire virtuelle.
- ☒ Ordonnancer les processus.
- ☐ Ordonnancer les opérations d'E/S.
- ☐ Ordonnancer les interruptions provoquées par les opérations d'E/S.

4. L'espace de swap sert à ?

- ☒ Stocker momentanément des processus qui ne peuvent tenir (pour des raisons de place) dans la mémoire principale (centrale).
- ☐ Sauvegarder les fichiers temporaires de l'utilisateur.
- ☐ Accélérer le démarrage du système en préchargeant les processus essentiels.
- ☐ Stocker des processus préemptés par le noyau.

5. Il y a deux liens physiques vers le fichier « f1 » qui sont « ph1 » et « ph2 » et un lien symbolique « sy1 ». Que se passe-t-il si nous supprimons le fichier « f1 » ?

- ☐ Nous ne pourrions pas accéder au fichier avec ph1 et ph2 mais avec sy1
- ☐ Nous pourrions accéder au fichier avec n'importe ph1, ph2 et sy1
- ☒ Nous pourrions toujours accéder au fichier avec ph1 et ph2 mais pas avec sy1
- ☐ Nous ne pourrions pas accéder au fichier avec ph1, ph2 et sy1

6. La valeur des permissions 652 est équivalente aux permissions :

- ☐ rw- rw- r--
- ☐ rwx r-x r-x
- ☒ rw- r-x -w-

7. Si un fichier possède les permissions `rwxr-x--x`, quel utilisateur peut le modifier ?

- ☐ Tout le monde
- ☒ Uniquement le propriétaire
- ☐ Le propriétaire et le groupe
- ☐ Uniquement le groupe

8. Dans un système d'exploitation, à quoi sert le « mask » ?

- ☐ À masquer les erreurs d'entrée/sortie du système.
- ☒ À définir les permissions par défaut lors de la création d'un fichier ou d'un répertoire.
- ☐ À crypter les données pour empêcher leur lecture non autorisée.
- ☐ À limiter l'accès aux fichiers en fonction de l'identité de l'utilisateur.

9. Quelle commande doit-on utiliser si on veut que tous les utilisateurs du système puissent exécuter le fichier prog ?

- ☐ chgrp bin /usr/local/bin/prog
- ☐ chown 755 /usr/local/bin/prog
- ☐ umask 0022 /usr/local/bin/prog
- ☒ chmod +x /usr/local/bin/prog

10. La commande 'cd \$HOME' est équivalente à :

- ☐ cd /
- ☐ cd /home
- ☐ cd ..
- ☒ cd ~

Exercice 2 (0.5 x 6 = 3pts)

1. Donner la commande shell qui permet d'ajouter un nouvel utilisateur.

`sudo useradd newuser`

2. Que fait la commande suivante : `gpasswd -a alice students`
qui est aussi équivalente à la commande : `usermod -aG students alice`

ajoute l'utilisateur **alice** au groupe **students**.

3. Dans quel fichier les comptes utilisateurs sont-ils définis ? Donner son chemin complet.

`/etc/passwd`

4. Que signifie SUID et en quoi ce droit spécial est-il utile ?

Le **SUID (Set User ID)** est un **droit spécial** attribué à un fichier exécutable sous Linux. Lorsqu'un utilisateur exécute un programme avec le bit **SUID** activé, le programme s'exécute avec les **privileges du propriétaire du fichier**, et non avec ceux de l'utilisateur qui l'exécute. ((Le SUID est particulièrement utile lorsque des utilisateurs doivent exécuter un programme qui nécessite des **droits élevés**, sans pour autant leur donner directement des **privileges administratifs**.)

5. Donner la commande shell qui permet de créer un lien symbolique sur un fichier f .

`ln -s f lien_symbolique`

- Donner la commande shell qui permet d'ajouter la permission d'écriture pour tous les membres du groupe propriétaire.

```
chmod g+w f
```

Exercice 3 (2 pts)

Écrire un script Shell qui prend en entrée le nom d'un fichier qui contient les quantités restantes en stock de différents produits (la quantité restante d'un produit par ligne), calcule la somme totale de ces quantités et affiche le résultat.

```
#!/bin/bash

# Vérifier si un fichier est fourni en argument
if [ $# -ne 1 ]; then
    echo "Usage : $0 <fichier_stock>"
    exit 1
fi

fichier="$1"

# Vérifier si le fichier existe
if [ ! -f "$fichier" ]; then
    echo "Erreur : Le fichier '$fichier' n'existe pas."
    exit 1
fi

# Initialisation de la somme
total_quantite=0

# Lire chaque ligne du fichier
while read -r quantite; do
    # Vérifier que la ligne contient un nombre valide
```

```

        if [[ "$quantite" =~ ^[0-9]+$ ]]; then
            total_quantite=$((total_quantite + quantite))
        else
            echo "Avertissement : Ligne ignorée ('$quantite' n'est
pas une quantité valide)."
        fi
    done < "$fichier"

# Affichage du résultat
echo "Total des quantités en stock : $total_quantite"

```

Exercice 4 (3 pts)

Écrire un script Shell qui calcule et affiche le nombre total de :

- Fichiers standard,
- Sous-répertoire,
- Fichiers exécutables

d'un répertoire qui sera fourni sous forme d'un paramètre ou bien saisis en cas d'absence de ce dernier.

```
#!/bin/bash
```

```
j=0
```

```
k=0
```

```
l=0
```

```
if [ "$#" -eq 0 ]; then
```

```
    echo "Saisir le répertoire"
```

```
    read -r rep
```

```
else
```

```
        rep=$1
fi

cd $rep

for i in *; do
    if [ -d "$i" ]; then
        echo "$i"
        let j=$j+1
    fi
    if [ -f "$i" ]; then
        echo $i
        let k=$k+1
    fi
    if [ -x "$i" ]; then
        echo $i
        let l=$l+1
    fi
done

echo "Il y a $j répertoires, $k fichiers et $l exécutable dans $rep"
```

Exercice 5 : Calculatrice simple (7 pts)

Écrire un script Shell sous le nom `calculatrice.sh` qui implémente une simple calculatrice en proposant un menu interactif et qui permet de :

1. Proposer les opérations suivantes : Addition (+), Soustraction (-), Multiplication (*), Division (/), Quitter (Q)
2. Demander à l'utilisateur de choisir une opération arithmétique ou Quitter
3. Demander à l'utilisateur de saisir deux opérandes.
4. Vérifier que les opérandes sont des nombres (sinon afficher une erreur et redemander la saisie).
5. Exécuter l'opération et afficher le résultat.
6. Gérer les cas particuliers (exemple : division par zéro).
7. Répéter l'affichage du menu et l'exécution des opérations jusqu'à ce que l'utilisateur décide de quitter.

Exemple d'exécution du script :

```
----- Calculatrice -----
1) Addition (+)
2) Soustraction (-)
3) Multiplication (*)
4) Division (/)
5) Quitter (Q)
Choisissez une option : 1
Entrez le premier nombre : 10
Entrez le deuxième nombre : 5
Résultat : 10 + 5 = 15

----- Calculatrice -----
1) Addition (+)
2) Soustraction (-)
3) Multiplication (*)
4) Division (/)
```

```
5) Quitter  
Choisissez une option : 4  
Entrez le premier nombre : 8  
Entrez le deuxième nombre : 0  
Erreur : Division par zéro impossible !
```

```
----- Calculatrice -----
```

```
1) Addition (+)  
2) Soustraction (-)  
3) Multiplication (*)  
4) Division (/)  
5) Quitter  
Choisissez une option : 5  
Au revoir !
```

```
#!/bin/bash
```

```
while true; do  
    # Affichage du menu  
    echo "----- Calculatrice -----"  
    echo "1) Addition (+)"  
    echo "2) Soustraction (-)"  
    echo "3) Multiplication (*)"  
    echo "4) Division (/)"  
    echo "5) Quitter"  
  
    echo -n "Choisissez une option : "  
    read choix
```

```

# Vérification si l'utilisateur veut quitter
if [ "$choix" -eq 5 ]; then
    echo "Au revoir !"
    exit 0
fi

# Vérification que l'option est valide
if [[ "$choix" -lt 1 || "$choix" -gt 4 ]]; then
    echo "Erreur : Option invalide !"
    continue
fi

# Saisie des opérandes
echo -n "Entrez le premier nombre : "
read a
echo -n "Entrez le deuxième nombre : "
read b

# Vérification que les entrées sont des nombres
if ! [[ "$a" =~ ^-?[0-9]+(\.[0-9]+)?$ && "$b" =~ ^-?[0-9]+(\.[0-9]+)?$ ]]; then
    echo "Erreur : Veuillez entrer des nombres valides !"
    continue
fi

# Exécution de l'opération
case $choix in

```



```
1) result=`expr $a + $b`; echo "Résultat : $a + $b =
$result" ;;

2) result=`expr $a - $b`; echo "Résultat : $a - $b =
$result" ;;

3) result=`expr $a * $b`; echo "Résultat : $a * $b =
$result" ;;

4)

    if [ "$b" == "0" ]; then

        echo "Erreur : Division par zéro impossible !"

    else

        result=`expr $a / $b`

        echo "Résultat : $a / $b = $result"

    fi

    ;;

esac

echo ""

done
```